MODULE 27 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Serving BI: Tools and Precomputation

Looker, Data Studio, Connected Sheets and third-party BI; the query cache, BI Engine, materialized views, scheduled queries and aggregate tables.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 27 of 45

admin@sthithapragna.com

Every dashboard click is a query. **Decide what to compute once, and what to compute on demand.**

Four ways to look at BigQuery

TOOL	BEST FOR	WHY
Looker	Governed reporting, such as the board	Metrics defined once, in a semantic model
Data Studio	Quick team dashboards	Self-service, fast to build, Pro for teams
Connected Sheets	Spreadsheet users, no SQL	Pivot billions of rows inside Sheets
Third-party BI	Tableau, Power BI and others	JDBC, ODBC or native connectors

CURRENCY CHECK

Looker Studio is called Data Studio again, since April 2026. Data Studio Pro is the paid tier.

Looker or Data Studio?

LOOKER

One governed definition

- Metrics defined once in LookML
- Version-controlled with Git
- Scheduled and embedded delivery
- Aggregate awareness uses summary tables

DATA STUDIO

Fast self-service

- Build a report in minutes
- Many data sources, little setup
- Pro adds team workspaces
- Each report holds its own logic

THE TIEBREAKER

One agreed definition of revenue for the board: Looker. A quick dashboard for a team: Data Studio.

BigQuery inside a spreadsheet

- Pivot tables, formulas and charts run over the **whole table**, as BigQuery queries.
- The sheet previews **500 rows**. Pivot results can reach **200,000 rows**.
- Extracts stop at **500,000 rows**. Over 50,000 rows, the cap is 5 million cells.
- Refreshes can be **scheduled**, and queries bill to the project you chose.

> THE TRAP
>
> Pulling millions of raw rows into Sheets fails. Summarise in BigQuery, and bring back the answer.

The ladder of precomputation

01

Query cache

Free. The same query again, within about a day.

02

BI Engine

In memory. Sub-second dashboard clicks.

03

Materialized views

BigQuery keeps aggregates fresh for you.

04

Scheduled tables

Any SQL, rebuilt on a timetable.

THE PATTERN

Climb only as far as you need. Each rung costs more effort, and covers more cases.

Free, but easy to miss

- Identical queries reuse results for **about 24 hours**, at no charge.
- Per user and project. Sharing across users needs **Enterprise** or Enterprise Plus.
- Missed when tables change, after streaming, or with a **destination table**.
- Missed with **CURRENT_TIMESTAMP**, row-level security, or even changed whitespace.

> THE TRAP
>
> A dashboard filter on CURRENT_DATE never hits the cache. Every click pays again.

Dashboards from memory

- A **reservation** of memory, per region, that caches the data dashboards use.
- Works for Looker, Data Studio, Tableau, Power BI, and any **JDBC or ODBC** tool.
- Choose **preferred tables**. Joins suit a fact table with up to four small dimensions.
- Not for external or Lakehouse tables, row-level security, JSON, or results over 1 GiB.

> THE PAYOFF
>
> No dashboard changes are needed. Queries are accelerated where they run.

Incremental or non-incremental?

INCREMENTAL

The default

- Aggregates such as SUM, COUNT, AVG, MIN, MAX
- Refresh reads only what changed
- Smart tuning rewrites matching queries
- Results reflect the latest base data

NON-INCREMENTAL

Allowed on request

- Most SQL, set with an option
- Every refresh recomputes everything
- Needs a max_staleness setting
- No automatic query rewrite

THE TIEBREAKER

Simple aggregates over a growing table: incremental. Complex SQL that can be a little stale: non-incremental.

Daily kWh for the customer app

```
1 CREATE MATERIALIZED VIEW marts.meter_daily_kwh
2 PARTITION BY DATE(reading_day)
3 CLUSTER BY meter_id
4 OPTIONS (enable_refresh = TRUE, refresh_interval_minutes = 60)
5 AS
6 SELECT TIMESTAMP_TRUNC(reading_ts, DAY) AS reading_day, meter_id,
7        SUM(kwh) AS kwh_total, COUNT(*) AS readings
8 FROM curated.readings_hh
9 GROUP BY reading_day, meter_id;
```

2 Partitioned to line up with the base table's reading-day partitions.

6-7 Simple aggregates, so the view refreshes incrementally.

REMEMBER

Queries still see readings that arrived after the last refresh. The view adds the changes on the fly.

Freshness, refresh and limits

- By default, results reflect the **latest base data**. Changes are read on the fly.
- **max_staleness**, from 30 minutes to 3 days, trades freshness for cost.
- Refresh starts within **5 minutes** of a change, no more than every **30 minutes** by default.
- Up to **100** per base table. No nesting. The query cannot be changed later.

> THE TRAP
>
> Questions that say a materialized view is always hours behind are usually wrong. Staleness is opt-in.

Materialized view or scheduled table?

MATERIALIZED VIEW

BigQuery maintains it

- Restricted SQL, mostly aggregates
- Incremental, always current
- Smart tuning finds it for you
- Nothing to schedule

SCHEDULED QUERY TABLE

You maintain it

- Any SQL, window functions, many joins
- As fresh as the last run
- Queries must name the table
- Dataform when steps depend on each other

THE TIEBREAKER

The SQL fits a materialized view: use one. Window functions or complex joins: a scheduled table.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
One definition of a metric for every report	Looker and LookML
Spreadsheet users, **billions of rows**	Connected Sheets
Sub-second dashboards, **no changes**	BI Engine reservation
Simple aggregates, **always current**	Incremental materialized view
Window functions, **changes nightly**	Scheduled query to a table
Same query, **cache never used**	Look for non-deterministic functions

THE PATTERN

Match the audience to the tool, then the query to the rung.

> A board dashboard ranks regions by revenue, using window functions over three large joined tables.
>
> The data changes once a night, when billing closes, yet every dashboard view scans 2 TB.

What should the team do?

A. Add all three tables to a BI Engine reservation as preferred tables

B. Create an incremental materialized view of the whole dashboard query

C. A scheduled query after billing closes, writing a results table

D. Turn off the query cache, so that the ranking is always fully fresh

◆ PAUSE AND DECIDE

What should the team do?

A **BI Engine**

Window functions over large joined tables are outside its limits.

B **Incremental view**

Window functions are not supported incrementally.

C **Scheduled table**

Runs once a night, as the data changes. Clicks read a tiny table.

D **No cache**

Makes every click cost more, and the data only changes nightly.

THE SIGNAL

Complex SQL over data that changes nightly: a scheduled query writing a small table.

> The customer app shows each meter's daily kWh, thousands of times an hour, from the huge half-hourly readings table.
>
> Readings that arrived minutes ago must appear, and analysts run the same daily totals against the base table by hand.

What should the team build?

A. An incremental materialized view, with smart tuning for analysts

B. A scheduled query every five minutes, overwriting a daily totals table

C. A logical view over the readings table that computes the daily totals

D. A nightly export of daily totals to Cloud Storage, behind an external table

◆ PAUSE AND DECIDE

What should the team build?

A Incremental view

Current to the minute, and analysts' queries are rewritten to use it.

B Five-minute schedule

Full rebuilds all day, and analysts still scan the base table.

C Logical view

Saves nothing. Every request recomputes from the base table.

D Nightly export

A day out of date, and slower to query.

THE SIGNAL

Simple aggregates that must stay current, queried many ways: an incremental materialized view.

> Larkspur's finance team works entirely in spreadsheets, and wants to pivot two billion readings by month and tariff without writing SQL.
>
> Their CSV exports fail beyond a million rows.

What should the team give them?

A. Exported CSV files in Cloud Storage, opened in Sheets one by one

B. Connected Sheets, with a pivot table over the BigQuery table

C. A Data Studio report with a table listing every single row

D. A Looker model, with a LookML developer assigned to each analyst

◆ PAUSE AND DECIDE

What should the team give them?

A CSV exports

The same row limits that already fail.

B Connected Sheets

The pivot runs in BigQuery over all rows. Sheets shows the result.

C Data Studio table

A dashboard, not a spreadsheet, and no row-level pivots.

D Looker per analyst

Heavy and costly for people who want a spreadsheet.

THE SIGNAL

Spreadsheet users, billions of rows, no SQL: Connected Sheets.

> Team dashboards in Data Studio sit on a 50 GB star schema: one fact table and three small dimension tables.
>
> Each click takes four to six seconds. Users want sub-second response, without anyone rebuilding the dashboards.

What should the team do?

A Rewrite each dashboard query as a scheduled query to its own table

B Move the four tables into Cloud SQL, which answers small reads faster

C Extend the dataset's time travel window to the full seven days

D A BI Engine reservation, with the star schema as preferred tables

◆ PAUSE AND DECIDE

What should the team do?

A **Scheduled tables**
Means rebuilding every dashboard, which the brief rules out.

B **Cloud SQL**
A transactional database, and every dashboard must be repointed.

C **Time travel**
Recovery, not speed.

D **BI Engine**
In-memory, sub-second, and no dashboard changes.

THE SIGNAL

Sub-second dashboards on a star schema, with no rebuilds: BI Engine.

Module 27 in five lines

- **Looker** for governed metrics, **Data Studio** for quick dashboards.
- **Connected Sheets** pivots billions of rows for spreadsheet users.
- The **cache** is free for a day, but misses on any change or CURRENT_DATE.
- **BI Engine** for sub-second dashboards, with no changes to them.
- **Materialized views** for current aggregates. **Scheduled tables** for anything else.

COMING UP

Next: module 28, fixing slow and expensive queries.